

CSE221: Algorithms

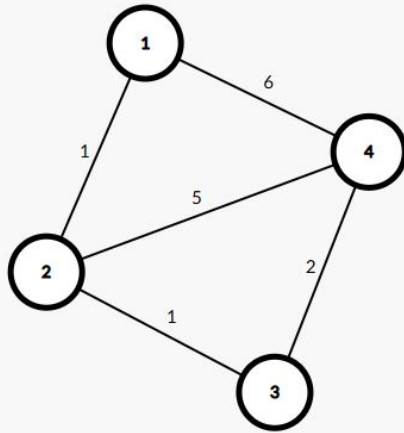
Shortest Path Algorithms

Prepared by: A B M Muntasir Rahman [MUNR]

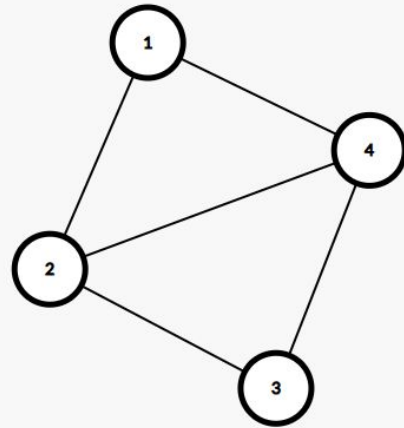


Inspiring Excellence

Weighted vs Unweighted Graph

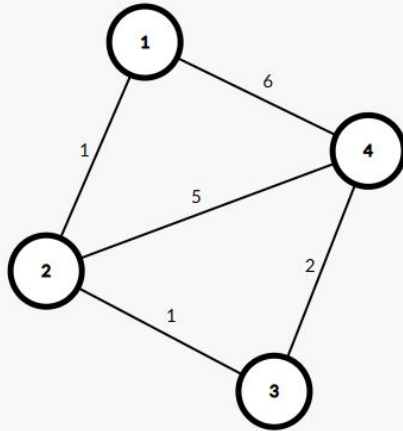


Weighted Graph



Unweighted Graph

Weighted Graph Shortest Path



Let, **1** is **source** and **4** is **destination**

Unweighted Shortest Path:

1 -> 4

Weighted Shortest Path:

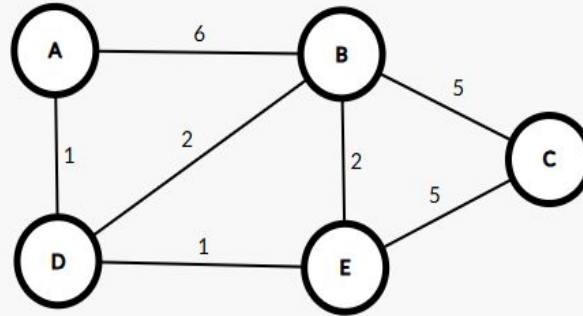
1 -> 2 -> 3 -> 4

Single Source Shortest Path Problem

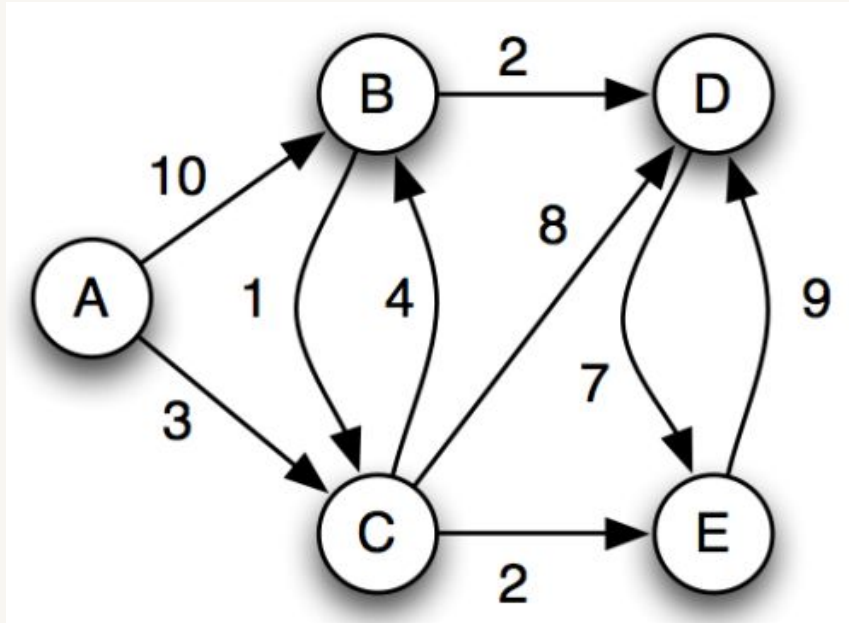
- We will discuss two general case algorithms:
 - ❖ **Dijkstra's Algorithm** (positive weights)
 - ❖ **Bellman - Ford Algorithm** (positive and negative weights)
- However, if the graph has equal weights for all the edges i.e. 1/2/3 so on, the problem can be solved by BFS traversal.

Dijkstra's Algorithm

- **Single Source Shortest Path (SSSP)** Algorithm i.e. given a source vertex, Dijkstra can find the **shortest path and path cost** to all the other connected vertices.
- Graph can be either **directed or undirected**. However, the graph must be **weighted with positive edge weights**.
- Similar to BFS.



Simulation



Visited

0	0	0	0	0
A	B	C	D	E

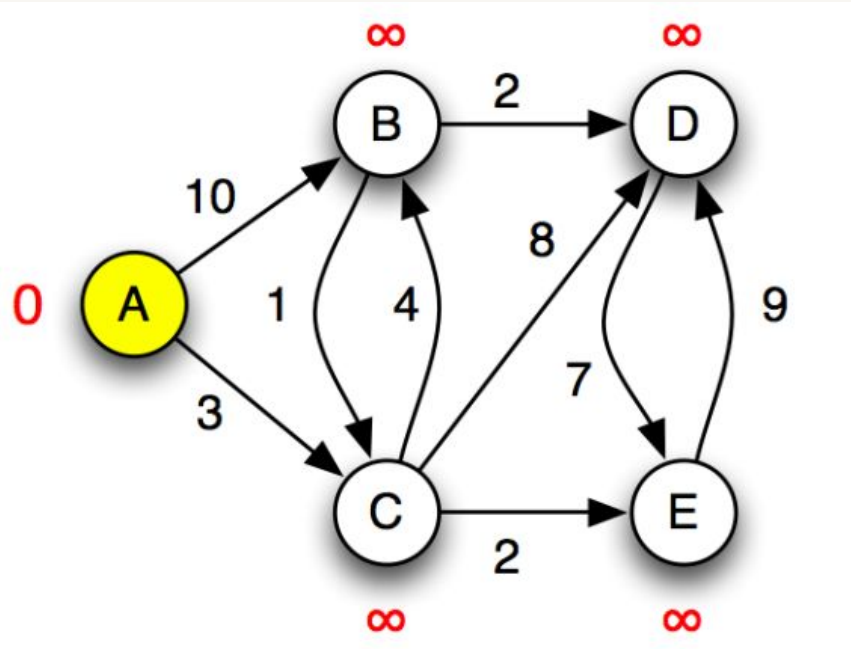
Distance

∞	∞	∞	∞	∞
A	B	C	D	E

Parent

N	N	N	N	N
A	B	C	D	E

Simulation



Visited

1	0	0	0	0
A	B	C	D	E

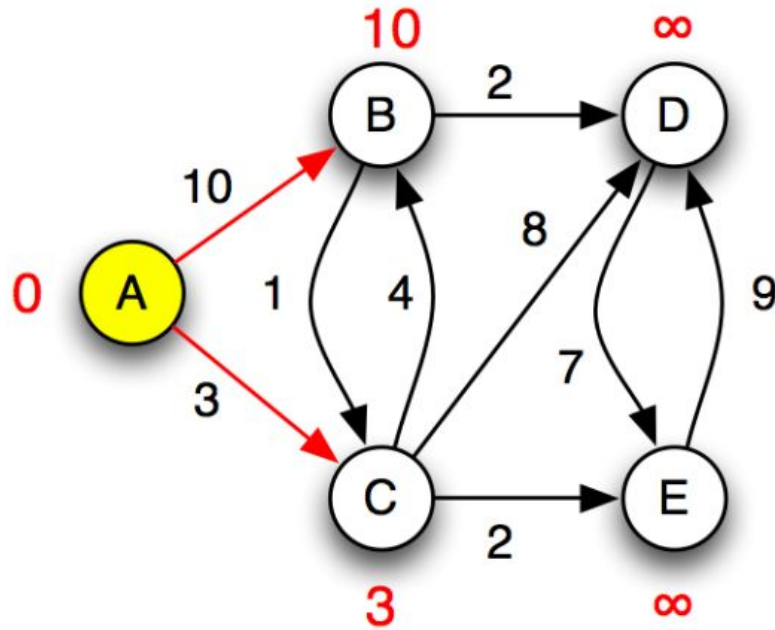
Distance

0	∞	∞	∞	∞
A	B	C	D	E

Parent

-1	N	N	N	N
A	B	C	D	E

Simulation



Visited

1	1	1	0	0
A	B	C	D	E

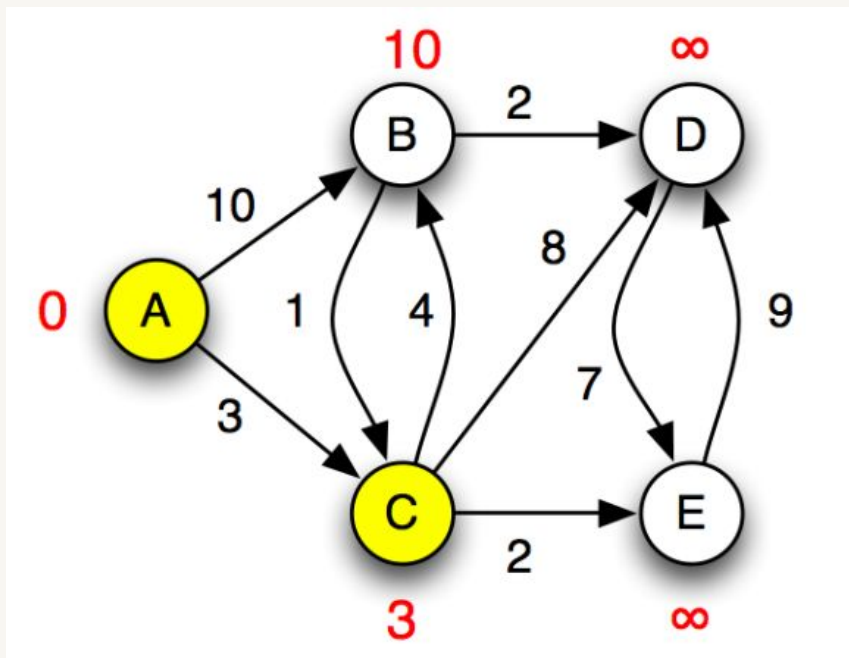
Distance

0	10	3	∞	∞
A	B	C	D	E

Parent

-1	A	A	N	N
A	B	C	D	E

Simulation



Visited

1	1	1	0	0
A	B	C	D	E

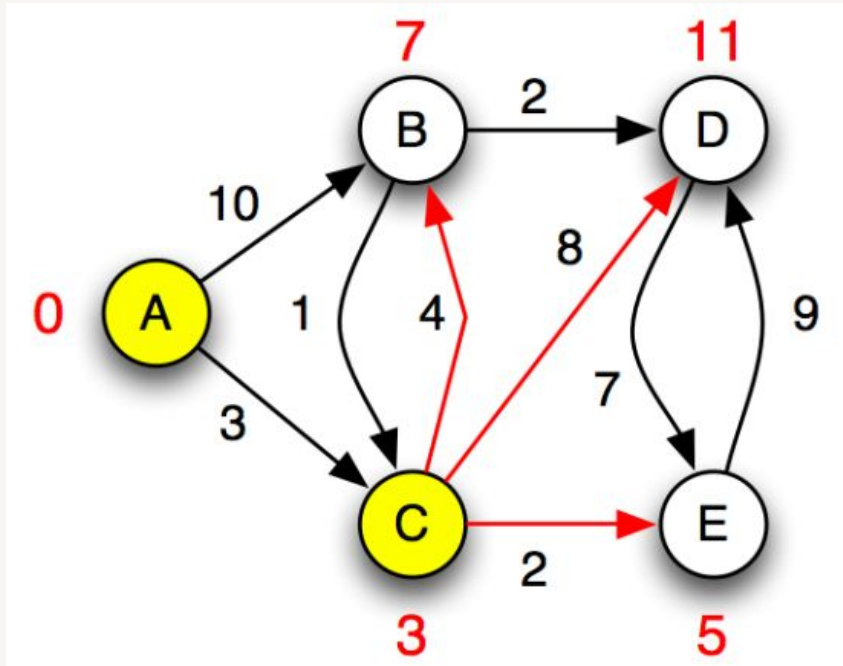
Distance

0	10	3	∞	∞
A	B	C	D	E

Parent

-1	A	A	N	N
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
A	B	C	D	E

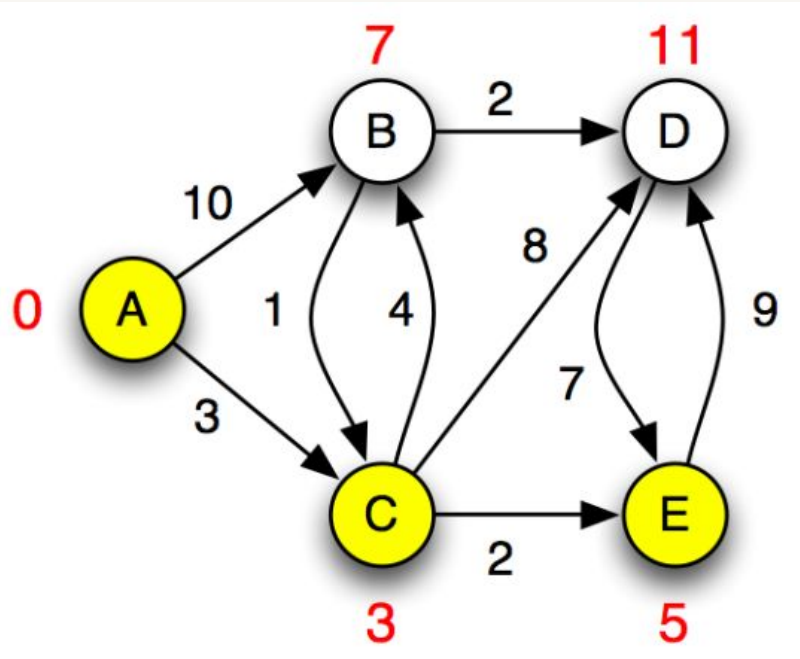
Distance

0	10 7	3	∞ 11	∞ 5
A	B	C	D	E

Parent

-1	A C	A	N C	N C
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
A	B	C	D	E

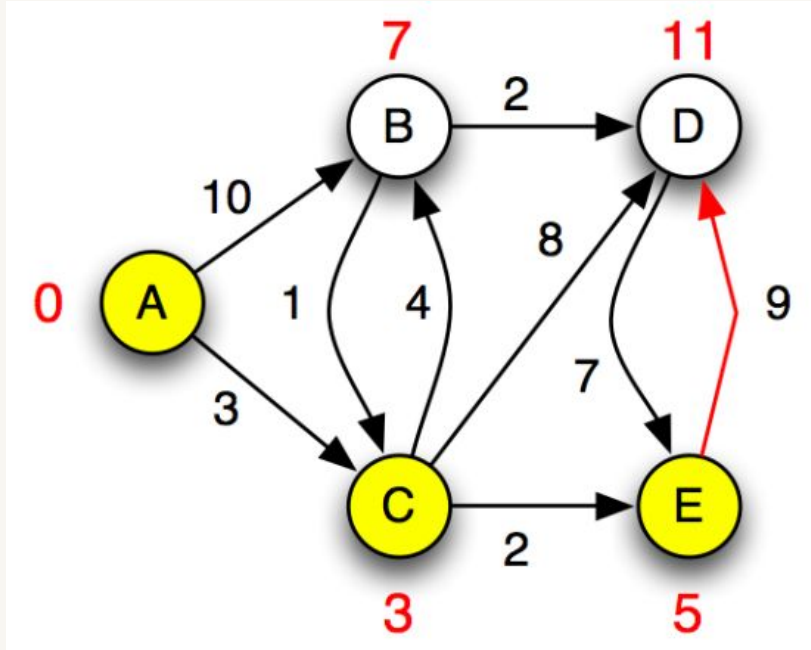
Distance

0	10 7	3	∞ 11	∞ 5
A	B	C	D	E

Parent

-1	A C	A	N C	N C
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
A	B	C	D	E

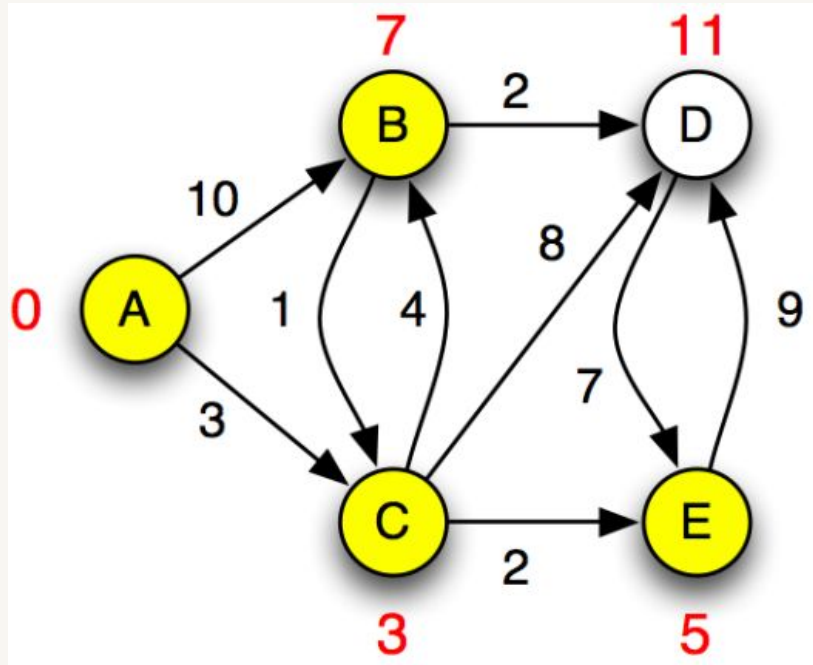
Distance

0	10 7	3	∞ 11	∞ 5
A	B	C	D	E

Parent

-1	A C	A	N C	N C
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
A	B	C	D	E

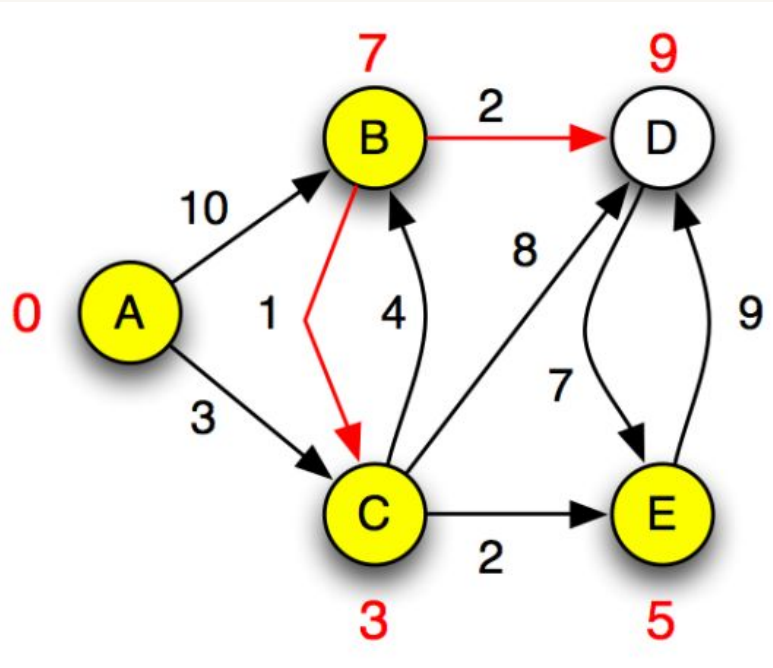
Distance

0	10 7	3	∞ 11	∞ 5
A	B	C	D	E

Parent

-1	A C	A	N C	N C
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
A	B	C	D	E

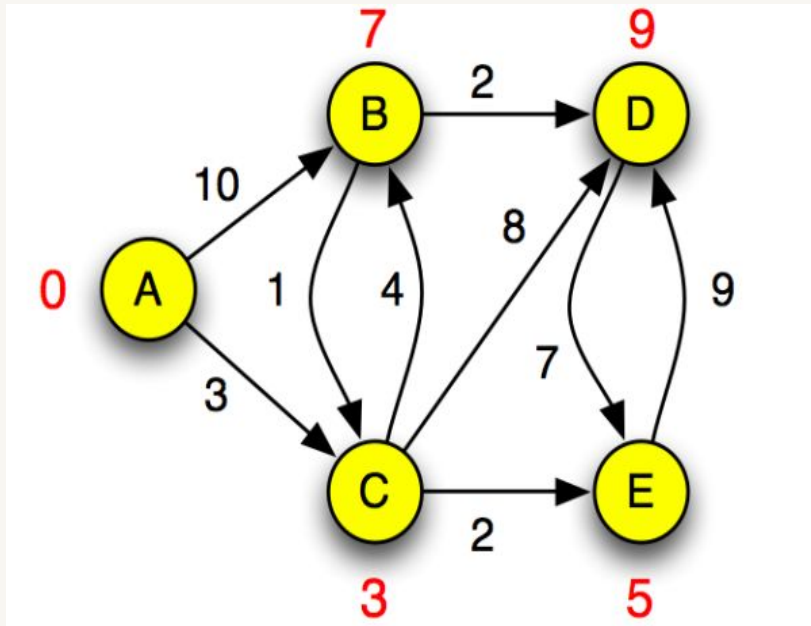
Distance

0	10 7	3	∞ 11 9	∞ 5
A	B	C	D	E

Parent

-1	A C	A	N C B	N C
A	B	C	D	E

Simulation



Visited

1	1	1	1	1
---	---	---	---	---

A B C D E

Distance

0	10 7	3	∞ 11 9	∞ 5
---	-----------------	---	--	----------------------------------

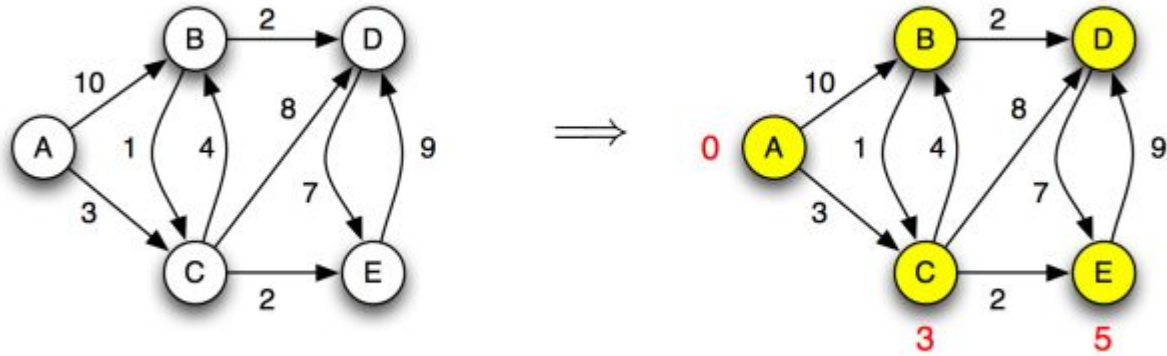
A B C D E

Parent

-1	A C	A	N C B	N C
----	----------------	---	-----------------------------	----------------

A B C D E

Simulation



Simulation

Visited

1	1	1	1	1
A	B	C	D	E

Distance

0	7	3	9	5
A	B	C	D	E

Parent

-1	C	A	B	C
A	B	C	D	E

What is the shortest distance between A to E?

Ans: Go to index E of distance array (5)

What is the shortest path from A to E?

Ans: Go to index E of parent array (C). Since C is not source, go to index C (A). Since A is the source, path will be **A -> C -> E**.

Pseudocode

Dijkstra(G, s)

$Q = []$, $parent = []$, $visited = []$, $d = []$

for each v in $V[G]$

$distance[v] = \infty$

$parent[v] = null$

$distance[s] = 0$

$Q = V[G]$

while Q is not empty:

$u = extract_min(Q)$

 for each vertex $v \in Adj[u]$ and $visited[v] \neq 1$:

 if $d[v] > d[u] + w(u, v)$

$d[v] = d[u] + w(u, v)$

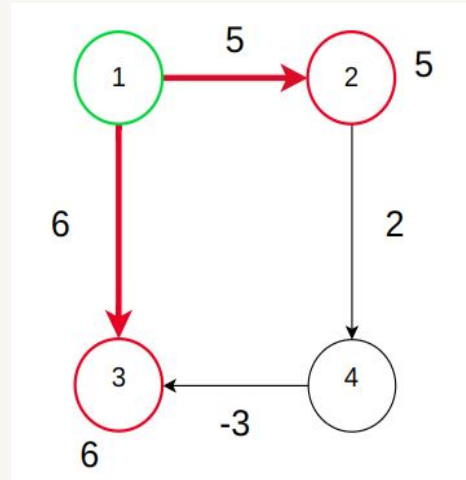
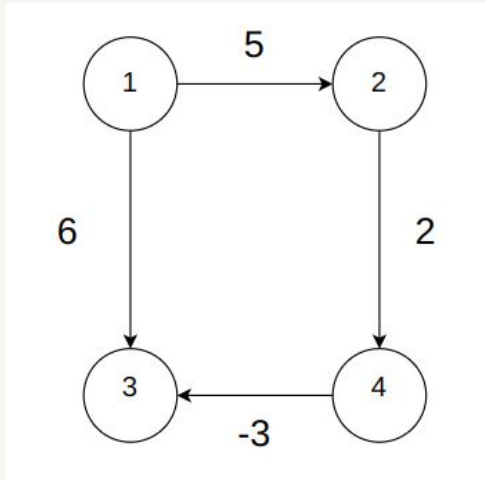
$parent[v] = u$

Relaxation of vertex v

$visited[u] = 1$ //Only update when all adjacent vertices are explored

Approach

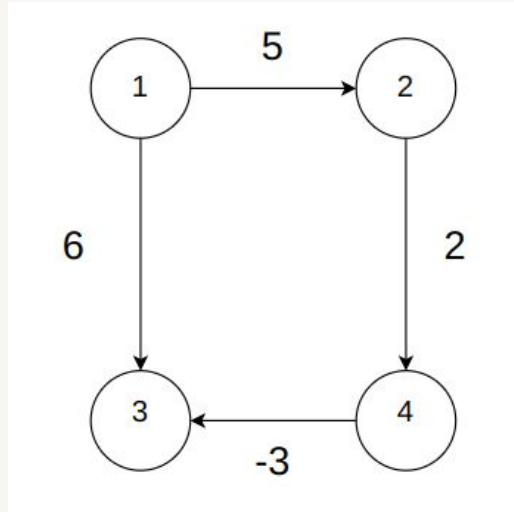
- Dijkstra follows the **Greedy Approach** to find the shortest path in a **weighted graph**. At every step of the solution, Dijkstra algorithm picks the vertices with the minimum distance from the source vertex (i.e. greedy choice).



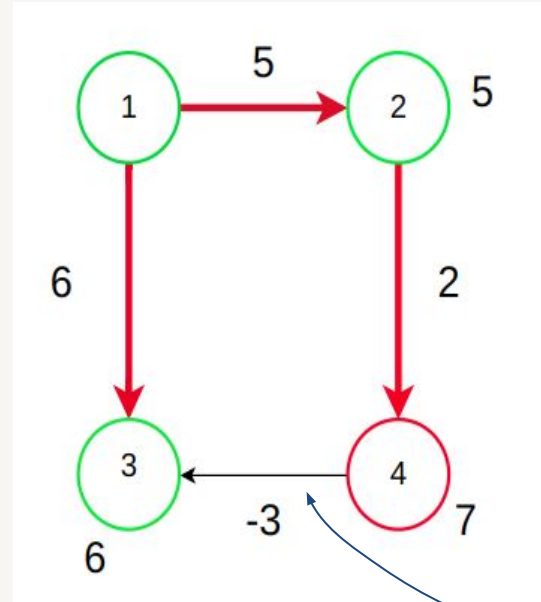
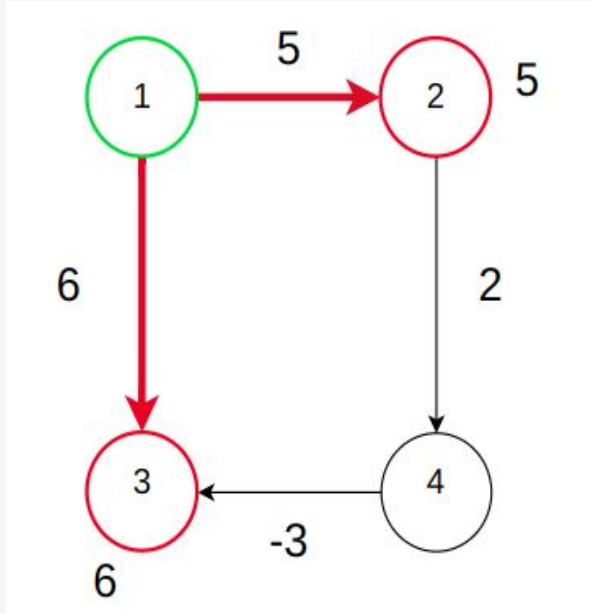
Starting from source vertex 1, both the edges (1, 2) and (1, 3) will be relaxed. Distance of vertex 2 will be 5 and distance of vertex 3 will be 6. For next step, Dijkstra will choose vertex 2 as it has the minimum distance (i.e. greedy approach)

Limitations of Dijkstra

- **Might** fail to find the shortest path in case of negative weights present in the graph.



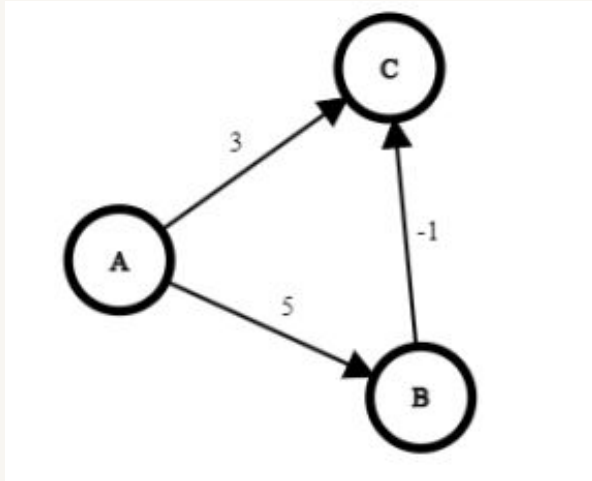
Limitations of Dijkstra



Dijkstra will not relax this edge because vertex 3 is already visited. Solution? **Bellman - Ford Algorithm!**

Limitations of Dijkstra

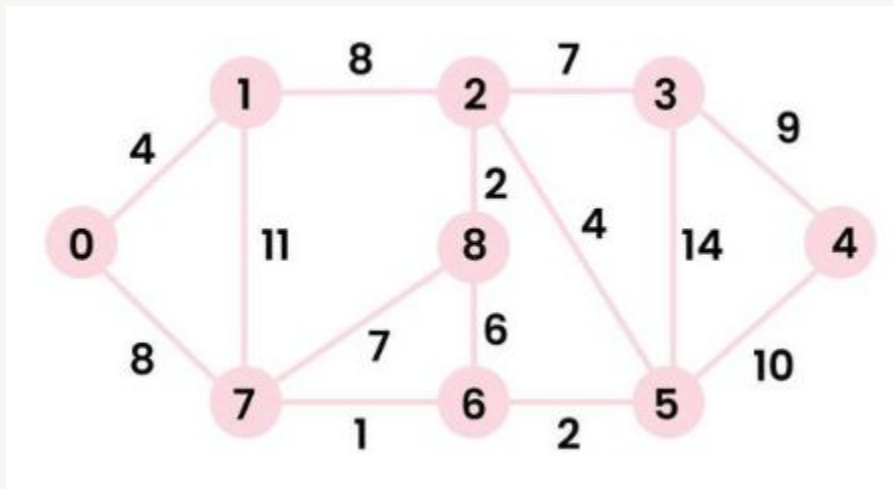
- For the below graph, Dijkstra will be able to find the shortest path given A as source even though negative weight is present.



Properties

- Time Complexity: **$O((V+E) \log V)$ using Min-Heap based Priority Queue**
 - However, if array / list implementation is used for Priority Queue, it is **$O(V^2)$** (where V = number of vertices).
- Space Complexity: **$O(V)$**
 - However, if graph representation is taken in consideration, it is **$O(V+E)$** .
- Follows **greedy approach**.
- Works on **directed and undirected graphs**.
- If the graph is **not connected**, it will not be able to visit all vertices.
- **Does not work if negative weights are present in the graph (except some exceptions).**

Practice



What is the shortest path and path distance from 1 to 4?

Reference

- Graph Algorithms by Mumit Khan
<https://drive.google.com/file/d/1MCbK6w6tSITZ3UrV9Na4-tPTolKXjO3D/view?usp=sharing>